

From Gradient Descent to SGD

GD, CG, Momentum, Nesterov, and Stochastic Methods — with Proofs

Dr. Robert Hildebrand

Grado Department of Industrial & Systems Engineering
Virginia Tech

Companion viewing — watch this first

“Why Momentum Really Works”

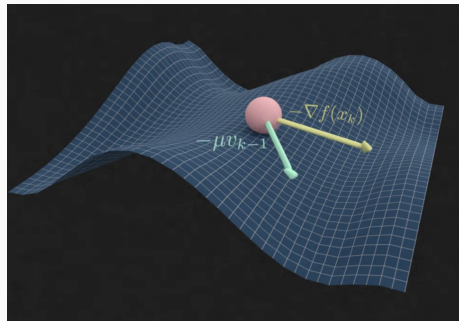
a 4-minute geometric explainer.

youtube.com/watch?v=iudXf5n_3ro

What you’ll see. A short visual tour of why a small momentum coefficient changes gradient descent from $O(1/k)$ to $O(1/k^2)$ on a narrow valley. The video uses the same quadratic-bowl picture we’ll use throughout today.

What to take away.

- “Looking ahead” before you compute the gradient is what makes momentum work.
- The optimal momentum coefficient depends on the condition number κ .



Still from the video. The pink ball rolls on a wavy loss surface. The yellow arrow is the new gradient $-\nabla f(\mathbf{x}_k)$; the green arrow is the prior velocity $-\mu \mathbf{v}_{k-1}$. The next step is their sum.

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

The problem and the notation we'll use

Problem.

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}).$$

Two assumptions on f .

- f is L -smooth: $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y}$.

Equivalent picture: f has a **quadratic upper bound** at every point.

- f is μ -strongly convex: $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Equivalent picture: f has a **quadratic lower bound** at every point.

Condition number. $\kappa := L/\mu \geq 1$. Large κ = narrow valley = hard.

Notation. Iterate $\mathbf{x}^k$. Optimum $\mathbf{x}^*$, optimal value $f^* = f(\mathbf{x}^*)$. Suboptimality $r_k := f(\mathbf{x}^k) - f^*$.

The descent lemma — the workhorse

Lemma (Descent lemma; Beck 2017, First-Order Methods in Optimization, Lemma 5.7)

If f is L -smooth, then for all $\mathbf{x}, \mathbf{y}$,

$$f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2.$$

Proof. For any $\mathbf{x}, \mathbf{y}$, define $\phi(t) := f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))$. Then $\phi'(t) = \nabla f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))^\top (\mathbf{y} - \mathbf{x})$, so

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) = \int_0^1 [\phi'(t) - \phi'(0)] dt.$$

Apply Cauchy–Schwarz and L -smoothness inside the integrand:

$$|\phi'(t) - \phi'(0)| \leq \|\nabla f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - \nabla f(\mathbf{x})\| \|\mathbf{y} - \mathbf{x}\| \leq t L \|\mathbf{y} - \mathbf{x}\|^2.$$

Integrate: $f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) \leq \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2.$

□

This single inequality drives every rate proof for the rest of the deck.

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

Gradient descent — recap

Algorithm. Step size $\eta = 1/L$:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \frac{1}{L} \nabla f(\mathbf{x}^k).$$

Per-step decrease.

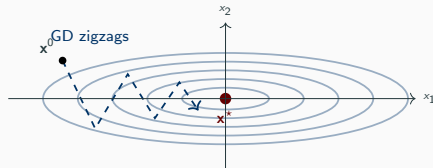
$$f(\mathbf{x}^{k+1}) \leq f(\mathbf{x}^k) - \frac{1}{2L} \|\nabla f(\mathbf{x}^k)\|^2 \quad [\text{Beck Lemma 10.4, } g \equiv 0].$$

Theorem (Beck, Theorems 10.21 and 10.29 with $g \equiv 0$)

Let f be L -smooth, $D = \|\mathbf{x}^0 - \mathbf{x}^*\|$. With $\eta = 1/L$:

- Convex: $f(\mathbf{x}^k) - f^* \leq \frac{LD^2}{2k}$.
- μ -strongly convex:
$$f(\mathbf{x}^k) - f^* \leq \left(1 - \frac{\mu}{L}\right)^k (f(\mathbf{x}^0) - f^*).$$

The catch. Both rates scale with $\kappa = L/\mu$. To halve the gap: $\sim \kappa \log 2$ iterations. **Big κ kills you.** Two routes around the wall: (1) for quadratics: conjugate gradient (next slide); (2) for general smooth convex: momentum / Nesterov (rest of deck).



Narrow valley. Each GD step is perpendicular to the contour at $\mathbf{x}^k$, so it largely undoes the previous step.

Conjugate gradient — recap

Setting. $\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$, with $\mathbf{A} \succ 0$. Equivalent to solving $\mathbf{A} \mathbf{x} = \mathbf{b}$.

Idea (Hestenes–Stiefel 1952). Build $\mathbf{A}$ -conjugate search directions $\mathbf{p}^0, \mathbf{p}^1, \dots$ on the fly from residuals $\mathbf{r}_k = \mathbf{b} - \mathbf{A} \mathbf{x}^k$. Conjugacy ($\mathbf{p}^{i\top} \mathbf{A} \mathbf{p}^j = 0$ for $i \neq j$) makes each step “orthogonal” to all previous ones in the $\mathbf{A}$ -norm.

Algorithm [Nocedal & Wright, Algorithm 5.2]. $\mathbf{p}^0 = \mathbf{r}_0$, then for $k = 0, 1, \dots$:

$$\alpha_k = \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}^k \top \mathbf{A} \mathbf{p}^k}, \quad \mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{p}^k, \quad \mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}^k, \quad \beta_{k+1} = \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}, \quad \mathbf{p}^{k+1} = \mathbf{r}_{k+1} + \beta_{k+1} \mathbf{p}^k.$$

Per-iteration cost: *one* matrix-vector product $\mathbf{A} \mathbf{p}^k$ — same as GD.

Theorem (Hestenes & Stiefel 1952; Nocedal & Wright, Theorems 5.3 and 5.5)

Let $\kappa = L/\mu$. Conjugate gradient on a quadratic terminates exactly in at most n steps, and after k iterations

$$\|\mathbf{x}^k - \mathbf{x}^*\|_A \leq 2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^k \|\mathbf{x}^0 - \mathbf{x}^*\|_A,$$

where $\|\mathbf{v}\|_A := \sqrt{\mathbf{v}^\top \mathbf{A} \mathbf{v}}$. Iterations to halve the error: $\sim \frac{1}{2} \sqrt{\kappa} \log 2$.

Compare GD: $(1 - 1/\kappa)^k$ vs. CG's $(1 - 2/\sqrt{\kappa})^k$. **For quadratic problems, use CG, not GD.**

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

Polyak's heavy-ball method (1964)

Idea. On a narrow valley, GD's gradient steps fight each other. Add a *momentum term* that remembers the previous step.

Algorithm.

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f(\mathbf{x}^k) + \beta (\mathbf{x}^k - \mathbf{x}^{k-1}).$$

Two hyperparameters: $\eta > 0$ and $\beta \in [0, 1)$.

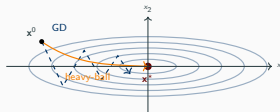
Velocity form. Carry a velocity vector $\mathbf{v}$:

$$\mathbf{v}^{k+1} = \beta \mathbf{v}^k - \eta \nabla f(\mathbf{x}^k), \quad \mathbf{x}^{k+1} = \mathbf{x}^k + \mathbf{v}^{k+1}.$$

Subtracting consecutive iterates recovers the boxed update.

Why “momentum”? Unrolling:

$\mathbf{v}^{k+1} = -\eta \sum_{j=0}^k \beta^{k-j} \nabla f(\mathbf{x}^j)$. The current step is an *exponentially weighted average* of all past gradients, with horizon $\sim 1/(1 - \beta)$. (Same form Adam, RMSProp, and SGD-with-momentum use in code.)

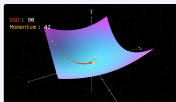


Same starting point, same step size. Heavy-ball's momentum smooths out the zigzag, “pushing” the iterate along the long axis.

Companion clip (a few seconds at the timestamp).

youtube.com/watch?v=MD2fY1p6QsQ&t=566s

“Who’s Adam and What’s He Optimizing?” — visual demo of SGD vs. momentum on a 2-D loss surface (right). The counter at top-left counts iterations: SGD takes ~ 90 steps, momentum takes ~ 42 . We’ll watch the segment around 9:26 in lecture.



Heavy-ball as a damped oscillator: the physical picture

Newton's equation for a ball rolling on a surface.

A point mass m on the surface of f in a viscous medium with friction coefficient γ :

$$m\ddot{\mathbf{x}}(t) + \gamma\dot{\mathbf{x}}(t) + \nabla f(\mathbf{x}(t)) = \mathbf{0}.$$

Discretize with timestep h , finite differences for $\ddot{\mathbf{x}}$ and $\dot{\mathbf{x}}$:

$$m \frac{\mathbf{x}^{k+1} - 2\mathbf{x}^k + \mathbf{x}^{k-1}}{h^2} + \gamma \frac{\mathbf{x}^k - \mathbf{x}^{k-1}}{h} + \nabla f(\mathbf{x}^k) = \mathbf{0}.$$

Solve for $\mathbf{x}^{k+1}$ and identify hyperparameters:

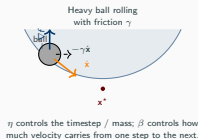
$$\boxed{\eta = \frac{h^2}{m}, \quad \beta = 1 - \frac{\gamma h}{m}.}$$

Reading the picture.

- β small (high friction): ball stops quickly; behaves like GD.
- $\beta \rightarrow 1$ (low friction): ball oscillates in the bowl indefinitely.
- β optimally tuned (*critical damping*): ball reaches the bottom fast without overshoot.

Why momentum sometimes hurts. On a non-convex landscape with cliffs, a heavy ball can roll past a sharp minimum because the kinetic energy doesn't dissipate fast enough.

Why this matters for tuning. If β is too large the algorithm overshoots and oscillates; too small and you lose the speedup. Most ML libraries default to $\beta = 0.9$ (i.e. "one-tenth friction per step"), which is roughly critically damped for typical problem conditioning.



Heavy-ball: rate on a quadratic

Setting. $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x} - \mathbf{b}^\top \mathbf{x}$, $A \succ 0$, eigenvalues in $[\mu, L]$, $\kappa = L/\mu$.

Optimal hyperparameters (Polyak):

$$\eta = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}, \quad \beta = \left(\frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}} \right)^2.$$

Theorem (Polyak 1964, Theorem 9 in Some methods of speeding up the convergence of iteration methods)

With these choices, on a quadratic f ,

$$\|\mathbf{x}^k - \mathbf{x}^*\| \leq C \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|\mathbf{x}^0 - \mathbf{x}^*\|$$

for some constant C . Same $\sqrt{\kappa}$ rate as conjugate gradient.

Proof idea. Diagonalize A . The recurrence $\mathbf{x}^{k+1} = \mathbf{x}^k - \eta A \mathbf{x}^k + \beta(\mathbf{x}^k - \mathbf{x}^{k-1})$ decouples per eigenmode. For eigenvalue λ the iteration matrix is

$$M_\lambda = \begin{pmatrix} 1 - \eta\lambda + \beta & -\beta \\ 1 & 0 \end{pmatrix}.$$

Optimal η, β make the spectral radius of M_λ equal to $\sqrt{\beta} = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ uniformly across $\lambda \in [\mu, L]$. $\square$

The catch. This rate is for *quadratics only*. On general convex L -smooth f , heavy-ball does *not* achieve the $\sqrt{\kappa}$ rate — it stays at $O(1/k)$.

When heavy-ball fails: the Lessard–Recht–Packard counterexample

Setup. Polyak's $\sqrt{\kappa}$ analysis works for global quadratics. Just outside that regime, momentum can blow up.

Theorem (Lessard, Recht, Packard 2016, SIAM J. Optim. 26(1))

There exists a 1-D, L -smooth, μ -strongly convex piecewise-quadratic f for which heavy-ball with Polyak's optimal (η, β) does not converge: iterates enter a periodic cycle.

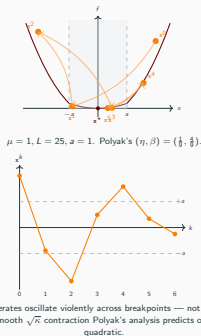
The construction. Piecewise-linear gradient with three slopes:

$$\nabla f(x) = \begin{cases} \mu x & |x| \leq a \\ L(x-a) + \mu a & x > a \\ L(x+a) - \mu a & x < -a \end{cases}$$

What goes wrong. When momentum carries $\mathbf{x}^k$ across a breakpoint, the gradient reports the old curvature; the step is tuned for the *wrong* slope, and overshoots.

Why this motivates Nesterov. Nesterov evaluates ∇f at the *look-ahead* point $\mathbf{y}^k$ instead of $\mathbf{x}^k$. The look-ahead reflects the curvature *where the iterate is going* — this kills the cycling on general convex L -smooth f .

Proof method. LRP frame momentum as a feedback control loop and analyze it with *integral quadratic constraints* (IQCs) from Lyapunov stability theory. The framework lets them *design* new momentum methods with provable rates — the start of a control-theoretic optimization line (Van Scoy–Freeman–Lynch 2018 “triple momentum”; Hu & Lessard 2017; Cyrus–Hu–Van Scoy–Lessard 2018).



Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

Nesterov's accelerated gradient method (1983)

The fix. Heavy-ball adds momentum *after* the gradient step. Nesterov takes the momentum step *first*, then evaluates the gradient at the look-ahead.

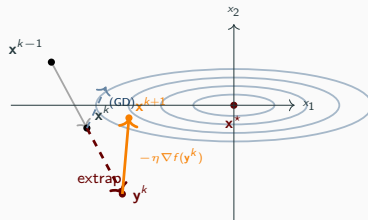
Algorithm. Init $\mathbf{x}^0$, $\mathbf{y}^0 = \mathbf{x}^0$, $t_0 = 1$. For $k = 0, 1, \dots$:

$$\mathbf{x}^{k+1} = \mathbf{y}^k - \frac{1}{L} \nabla f(\mathbf{y}^k)$$

$$t_{k+1} = \frac{1}{2} \left(1 + \sqrt{1 + 4t_k^2} \right)$$

$$\mathbf{y}^{k+1} = \mathbf{x}^{k+1} + \frac{t_k - 1}{t_{k+1}} (\mathbf{x}^{k+1} - \mathbf{x}^k)$$

(grad step from look-ahead; scheduled momentum; new look-ahead)



Maroon: extrapolated $\mathbf{y}^k$. **Orange:** gradient at $\mathbf{y}^k$ and the new iterate $\mathbf{x}^{k+1}$. **Blue dashed:** what GD would have done.

Three things to notice.

1. Same per-iteration cost as GD: *one* gradient evaluation, at $\mathbf{y}^k$.
2. t_k grows like $k/2$, so the momentum coefficient $(t_k - 1)/t_{k+1} \rightarrow 1$.
3. $\mathbf{y}^k$ is *not* a candidate iterate. It's just where we sample the gradient.

Nesterov: the $O(1/k^2)$ rate

Nesterov 1983; Beck–Teboulle 2009. Beck, *First-Order Methods in Optimization*, Theorem 10.34.

Theorem (Nesterov 1983, Soviet Math. Dokl. 27; Beck & Teboulle 2009 (FISTA), SIAM J. Imaging Sci. 2(1); Beck 2017, Theorem 10.34)

Let f be convex and L -smooth. Nesterov's method satisfies

$$f(\mathbf{x}^k) - f^* \leq \frac{2L \|\mathbf{x}^0 - \mathbf{x}^*\|^2}{(k+1)^2}.$$

Proof skeleton. Following Beck §10.7.2, define the potential

$$\Phi_k := \frac{2}{L} t_{k-1}^2 (f(\mathbf{x}^k) - f^*) + \|t_{k-1} \mathbf{x}^k - (t_{k-1} - 1) \mathbf{x}^{k-1} - \mathbf{x}^*\|^2.$$

Key lemma (Beck Lemma 10.33 + the derivation around eqs. 10.40–10.42). The recursion

$t_{k+1} = \frac{1}{2}(1 + \sqrt{1 + 4t_k^2})$ is exactly the value that makes Φ_k **non-increasing**:

$$\Phi_{k+1} \leq \Phi_k.$$

The proof combines (i) the descent lemma applied at the look-ahead $\mathbf{y}^k$, (ii) convexity of f at $\mathbf{y}^k$, and (iii) the algebraic identity $t_{k+1}^2 - t_{k+1} = t_k^2$ (which is the recursion's defining equation).

Since $\Phi_0 \leq \|\mathbf{x}^0 - \mathbf{x}^*\|^2$ and Lemma 10.33 gives $t_{k-1} \geq (k+1)/2$, monotonicity gives

$$\frac{2}{L} t_{k-1}^2 (f(\mathbf{x}^k) - f^*) \leq \Phi_k \leq \Phi_0 \leq \|\mathbf{x}^0 - \mathbf{x}^*\|^2,$$

so $f(\mathbf{x}^k) - f^* \leq \frac{L}{2t_{k-1}^2} \|\mathbf{x}^0 - \mathbf{x}^*\|^2 \leq \frac{2L}{(k+1)^2} \|\mathbf{x}^0 - \mathbf{x}^*\|^2$. □

The whole proof is two pages of careful algebra in Beck §10.7.2. The non-obvious step is guessing the right Φ_k — this is what Nesterov's “estimating sequences” technique systematically produces.

Why $O(1/k^2)$ is optimal: a lower bound

Theorem (Nemirovski–Yudin 1983; Nesterov 2004 §2.1.2)

For every dimension n and every $k \leq (n-1)/2$, there exists a convex L -smooth function f such that any algorithm whose iterates lie in $\mathbf{x}^0 + \text{span}\{\nabla f(\mathbf{x}^0), \nabla f(\mathbf{x}^1), \dots, \nabla f(\mathbf{x}^{k-1})\}$ satisfies

$$f(\mathbf{x}^k) - f^* \geq \frac{3L \|\mathbf{x}^0 - \mathbf{x}^*\|^2}{32(k+1)^2}.$$

The hard function. A specific quadratic with tridiagonal Hessian (“Nesterov’s worst function in the world”):

$$f(\mathbf{x}) = \frac{L}{4} \left[\frac{1}{2} x_1^2 + \frac{1}{2} \sum_{i=1}^{n-1} (x_{i+1} - x_i)^2 + \frac{1}{2} x_n^2 - x_1 \right].$$

What this says. No method that builds iterates from past gradient information can do better than $O(1/k^2)$ in the worst case. So the gap between GD’s $O(1/k)$ and Nesterov’s $O(1/k^2)$ is the *whole story* for first-order convex smooth optimization — there is no faster method.

Caveat. The lower bound assumes $k \leq (n-1)/2$, i.e. fewer iterations than half the dimension. For $k > n$ on quadratics, CG terminates exactly — the lower bound doesn’t apply.

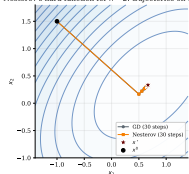
Nesterov's hard function visualized: $n = 2$ case

For $n = 2$, the worst function specializes to

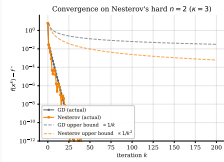
$$f(x_1, x_2) = \frac{L}{4} [x_1^2 - x_1 x_2 + x_2^2 - x_1], \quad A = \frac{L}{4} \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}, \quad \mathbf{x}^* = \left(\frac{2}{3}, \frac{1}{3}\right).$$

Hessian eigenvalues: $L/4$ and $3L/4$. Effective condition number $\kappa = 3$ (mild — the worst-case nature shows up only as n grows). The contours are tilted ellipses; GD zigzags, Nesterov glides.

Nesterov's hard function for $n = 2$; trajectories on contours



Same starting point $\mathbf{x}^0 = (-1, 1.5)$. GD (gray circles) and Nesterov (orange squares) both run for 30 iterations with $\eta = 1/L_{\max}(A)$. Nesterov's path bends toward the optimum and converges in a few iterations.



Suboptimality $f(\mathbf{x}^k) - f^*$ on a semilog axis, for 200 iterations. Solid: actual runs. Dashed: theoretical bounds $\frac{LD^2}{2k}$ (GD) and $\frac{3LD^2}{k+3}$ (Nesterov). The actual GD curve hugs the dashed sublinear envelope; Nesterov beats its $1/k^2$ envelope easily because $\kappa = 3$ is small.

What the picture buys you.

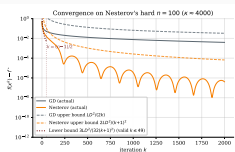
- The contour ellipses tilt because the off-diagonal $-x_1 x_2$ couples the coordinates — this coupling is what makes the function “hard” for GD: the gradient at $\mathbf{x}^k$ doesn't point at $\mathbf{x}^*$.
- For larger n (say $n = 100$), the eigenvalue ratio $\lambda_{\max}/\lambda_{\min} \approx 4/(\pi/n)^2 \sim 4n^2/\pi^2$ blows up, κ becomes large, and the lower bound bites: $O(1/k^2)$ is the best any first-order method can do.

Code. The trajectories and convergence curves are produced by `figures/make_nesterov_hard.py` — direct simulation of GD and Nesterov on this exact A, b . No tuning, no shortcuts. Try changing n in the script to see the lower-bound regime emerge.

Nesterov's hard function: $n = 100$ — the lower bound bites

For $n = 100$ the tridiagonal Hessian has eigenvalues $\lambda_i = (L_*/4)(2 - 2 \cos \frac{i\pi}{n+1})$, $i = 1, \dots, 100$. The smallest eigenvalue is $\sim L_*/(n+1)^2$ and the largest is $\sim L_*$, so

$\kappa \approx 4(n+1)^2/\pi^2 \approx 4,100$

 This is where the worst-case lower bound actually bites.

$n = 100$, 2000 iterations, $x^0 = 0$. Solid: actual GD (gray) and Nesterov (orange). Dashed: theoretical upper bounds. Dotted maroon: the Nemirovski–Yudin lower bound, valid only for $k \leq (n-1)/2 = 49$.

What to read off the plot.

- GD converges *slowly*: after 2000 iterations, suboptimality is still $\sim 4 \cdot 10^{-3}$. The condition number $\kappa \approx 4000$ means GD takes $\sim \kappa = 4000$ iterations just to halve the gap once strong convexity kicks in.
- Nesterov reaches $\sim 4 \cdot 10^{-7}$ in the same 2000 iterations — four orders of magnitude better.
- In the first 49 iterations, both Nesterov's actual curve and its upper bound are sandwiched against the lower bound (dotted maroon). **There is essentially no room left for improvement** — this is what “optimal” means.
- Past $k = 49$, the lower bound stops applying (Nemirovski–Yudin's argument is finite-dimensional). After that, Nesterov can keep accelerating; on this particular problem you can see the curve drop sharply once the strong convexity at the bottom of the spectrum becomes effective.

The take-away. For modest n (the $n = 2$ slide), Nesterov's $1/k^2$ envelope is loose — the algorithm beats it because $\kappa = 3$ is mild. For large n (this slide), the upper and lower bounds collapse onto each other in the first $\sim n/2$ iterations, and Nesterov sits exactly between them.

The hard function isn't a curiosity — it's a tight constraint.

The thirty-year mystery: why does Nesterov's method work?

The puzzle. Nesterov 1983 proved both (i) the rate $O(1/k^2)$ and (ii) the matching lower bound. The algorithm was *provably optimal* the day it appeared. But the proof used Nesterov's "estimating sequences" — a clever telescoping construction that offers little intuition for *why* the look-ahead step achieves the rate. The t_k recursion $t_{k+1} = (1 + \sqrt{1 + 4t_k^2})/2$ looked pulled from a hat. For three decades, people used the method because it worked, not because it was understood. Several papers, all post-2014, finally cracked it.

Four mutually-reinforcing perspectives.

- **Continuous-time ODE limit** [Su, Boyd, Candès 2014, NeurIPS / JMLR 2016]. As the step size shrinks, Nesterov's iteration is the discretization of the second-order ODE

$$\ddot{X}(t) + \frac{3}{t} \dot{X}(t) + \nabla f(X(t)) = 0.$$

This is a *damped oscillator with vanishing friction*. The constant 3 is exactly what makes the energy decay as $1/t^2$. The whole rate proof becomes a Lyapunov-function calculation on the ODE.

- **Linear coupling of gradient and mirror descent** [Allen-Zhu & Orecchia 2014, ITCS 2017]. Nesterov's algorithm is exactly the convex combination of (i) gradient descent (which exploits primal smoothness) and (ii) mirror descent (which exploits dual averaging via gradient summing). The look-ahead point y^k is the coupling parameter. The proof is a one-page summation.
- **Geometric construction via covering balls** [Bubeck, Lee, Singh 2015]. At each step, intersect two balls (one centered at the current iterate, one at the cumulative-gradient point) and pick the smallest covering ball. The center is the next iterate. The t_k recursion drops out of the ball-radius bookkeeping. *No explicit momentum is ever written down.*
- **Variational / Lagrangian perspective** [Wibisono, Wilson, Jordan 2016, PNAS]. A continuous-time Lagrangian $\mathcal{L}(X, \dot{X}, t)$ has Nesterov's ODE as its Euler-Lagrange equation. Different smoothness exponents give a *continuum* of accelerated schemes; Nesterov is one point on this continuum.

Insight worth carrying. A famous example of a result that was technically settled *decades* before it was understood. We now have four mutually-reinforcing pictures, none of which is Nesterov's original. The take-away for first-order method design: when in doubt, look for the underlying ODE, the underlying duality, or the underlying geometry — the algorithm is always richer than its discrete-time recursion.

Further reading. Bubeck's monograph *Convex Optimization: Algorithms and Complexity* (2015) has a one-chapter narrative tour through these perspectives. Sébastien's blog at blogs.princeton.edu/imahandit has accessible essays on each. The Su-Boyd-Candès paper is the most readable starting point if you have one afternoon.

Beyond Nesterov: Lan's universal and parameter-free methods

The pain points with classical Nesterov.

- Needs to know L in advance (used in step size $\eta = 1/L$).
- Needs to know μ in advance for the strongly-convex variant (different algorithm than the convex one).
- One algorithm per smoothness class (smooth-Lipschitz, Hölder, non-smooth) — the user picks before running.

Universal gradient method [Nesterov 2015, *Math. Programming* 152, "Universal gradient methods for convex optimization problems"]. A single accelerated method, run with backtracking line search on L , achieves the optimal rate across the entire **Hölder-smoothness spectrum**:

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L_\nu \|\mathbf{x} - \mathbf{y}\|^\nu, \quad \nu \in [0, 1].$$

- $\nu = 1$: Lipschitz gradient (classical case) — gets $O(1/k^2)$.
- $\nu = 0$: bounded gradient (non-smooth, $\|\mathbf{x}\|_1$ -type) — gets $O(1/\sqrt{k})$.
- Intermediate ν : smooth interpolation, automatically. **The user picks neither ν nor L_ν .**

Parameter-free accelerated proximal gradient [Lan 2020, *First-Order and Stochastic Optimization Methods for Machine Learning*, Springer, Ch. 3–4]. A single algorithm with adaptive backtracking on both L and μ achieves

$$f(\mathbf{x}^k) - f^* \leq \begin{cases} O(LD^2/k^2) & \text{if convex } (\mu = 0) \\ O\left(LD^2 \exp(-c\sqrt{\mu/L}k)\right) & \text{if } \mu\text{-strongly convex} \end{cases}$$

without any a-priori knowledge of L , μ , or $D = \|\mathbf{x}^0 - \mathbf{x}^*\|$. "Set and forget."

Insight. Lan's framework is the principled answer to the question "can we make accelerated methods as practical as L-BFGS?" — the algorithm *learns* the local geometry from gradient evaluations, the way a quasi-Newton method does. The cost: $\sim 2\times$ gradient evaluations per iteration in the worst case (the backtracking probe).

See also, d'Aspremont, Scieur & Taylor (2021), *Acceleration Methods*, Found. & Trends in Optimization 5(1–2). Modern survey covering universal methods, restart schemes, and the connection to Anderson acceleration. Lan's textbook is the most complete single source — the parameter-free chapter generalizes to stochastic gradients with unknown noise level σ , which connects directly to the SGD section later in this deck.

AC-FGM (2025): a three-step parameter-free accelerated method

Li & Lan, *Math. Programming* 2025, "A simple uniformly optimal method without line search for convex optimization"
(arXiv:2310.10082).

Goal. Get the optimal $O(1/k^2)$ rate on convex L -smooth f without knowing L , without a line search, and without an a-priori bounded domain. Just gradient evaluations.

The Auto-Conditioned Fast Gradient Method (AC-FGM) maintains three coupled sequences:

- $\{z_t\}$ — the new **auxiliary iterate** (a single mirror step).
- $\{y_t\}$ — the **prox-center** sequence (a weighted average of past z 's).
- $\{x_t\}$ — the **gradient-evaluation / output** sequence (another weighted average of past z 's).

Three-step update at iteration t . For composite $f + h$ with h closed convex (cheap prox):

$$\begin{aligned} z_t &= \arg \min_z \left\{ \eta_t \langle \nabla f(x_{t-1}), z \rangle + h(z) + \frac{1}{2} \|z - y_{t-1}\|^2 \right\} && \text{(mirror step from prox-center)} \\ y_t &= (1 - \theta_t) y_{t-1} + \theta_t z_t && \text{(update prox-center)} \\ x_t &= (1 - \tau_t) x_{t-1} + \tau_t z_t && \text{(update gradient-eval / output point)} \end{aligned}$$

The step sizes η_t, θ_t, τ_t are computed *auto-conditioned* on local curvature from finite gradient differences:

$$\eta_t \sim \frac{\|x_{t-1} - x_{t-2}\|}{\|\nabla f(x_{t-1}) - \nabla f(x_{t-2})\|}.$$

This is a local Lipschitz estimate — replaces $1/L$ in classical Nesterov.

Theorem (Li & Lan 2025, Math. Programming, Theorem 1)
On convex L -smooth f , without any prior knowledge of L or domain diameter, AC-FGM satisfies

$$f(x_k) - f^* \leq O\left(\frac{L \|x^0 - x^*\|^2}{k^2}\right).$$

The rate is uniformly optimal across the Hölder smoothness spectrum (same construction adapts automatically).

The structural innovation. Classical Nesterov has two sequences (x, y) ; FISTA has two; Lan's earlier parameter-free APG has three with backtracking. AC-FGM has three sequences *without backtracking* — the prox-center role and the gradient-evaluation role are split into two separate weighted averages of the auxiliary z . That separation is what kills the line-search dependence.

Stochastic extension. Li & Lan (2026), "Stochastic Auto-conditioned Fast Gradient Methods with Optimal Rates" (arXiv:2604.06526), extends AC-FGM to stochastic gradients with unknown noise level. **Code.** github.com/y1432/AC-FGM-Implementation.

Strongly convex case — a constant momentum coefficient

For μ -strongly convex L -smooth f , replace the scheduled momentum with the constant choice

$$\beta = \frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}}, \quad \mathbf{y}^k = \mathbf{x}^k + \beta(\mathbf{x}^k - \mathbf{x}^{k-1}), \quad \mathbf{x}^{k+1} = \mathbf{y}^k - \frac{1}{L} \nabla f(\mathbf{y}^k).$$

Theorem (Nesterov 2004 §2.2.1; Beck Theorem 10.42 (V-FISTA))

With this choice,

$$f(\mathbf{x}^k) - f^* \leq \left(1 - \sqrt{\mu/L}\right)^k \left(f(\mathbf{x}^0) - f^* + \frac{\mu}{2} \|\mathbf{x}^0 - \mathbf{x}^*\|^2\right).$$

Compare again.

Method	Convex (suboptimality)	μ -strongly convex
Gradient descent	$O(LD^2/k)$	$(1 - 1/\kappa)^k$
Heavy-ball (quadratics only)	—	$((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^{2k}$
Conjugate gradient (quadratics only)	—	$((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^k$
Nesterov (general)	$O(LD^2/k^2)$	$(1 - 1/\sqrt{\kappa})^k$

Take-away. Across the row marked “Nesterov” is the right column to live in for general first-order optimization: the same algorithm, in two regimes, achieves the optimal rate.

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

Empirical risk minimization and loss functions

Setting. Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$. Predictor $h_\theta(\mathbf{x})$ with parameters $\theta \in \mathbb{R}^p$.

A **loss function** $\ell(h_\theta(\mathbf{x}), y) \geq 0$ measures how bad the prediction is on one example. The **empirical risk** averages over the training set:

$$F(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(h_\theta(\mathbf{x}_i), y_i) + R(\theta),$$

optionally with a regularizer R (e.g. $R(\theta) = \frac{\lambda}{2} \|\theta\|^2$). The whole job of training is $\min_\theta F(\theta)$.

Three loss functions to keep in your head.

Loss	Formula	Use	Convex?
Squared loss	$\ell = \frac{1}{2}(y - h_\theta(\mathbf{x}))^2$	Linear / ridge regression	Yes, smooth
Logistic loss	$\ell = \log(1 + e^{-y h_\theta(\mathbf{x})})$	Logistic regression	Yes, smooth
Hinge loss	$\ell = \max\{0, 1 - y h_\theta(\mathbf{x})\}$	Support vector machine	Yes, non-smooth

Smooth vs. non-smooth. Squared and logistic losses are differentiable everywhere — gradient methods apply directly. Hinge loss has a kink at $y h_\theta(\mathbf{x}) = 1$ — needs subgradient methods or a smooth surrogate.

Why optimization? ML training = choose a hypothesis class h_θ , choose a loss ℓ , then *minimize* the empirical risk $F(\theta)$. The minimization is the optimization problem. Most modern ML training algorithms are first-order methods — exactly the ones in this deck.

Two concrete applications: what $\mathbf{x}_i$, y_i , and ℓ actually are

Application 1: predicting house prices (regression).

One training example i = one house sold. The feature vector and the label:

$$\mathbf{x}_i = \begin{pmatrix} \text{sqft} \\ \text{\#beds} \\ \text{\#baths} \\ \text{age} \\ \text{ZIP code one-hot} \end{pmatrix} \in \mathbb{R}^p, \quad y_i \in \mathbb{R}_+ = \text{sale price.}$$
$$\begin{pmatrix} \text{count("viagra")} \\ \text{count("free")} \\ \text{count("meeting")} \\ \text{has_attachment} \\ \text{sender domain features} \\ \vdots \end{pmatrix} \in \mathbb{R}^p, \quad y_i \in \{-1, +1\}.$$

Concretely, $\mathbf{x}_1 = (1500, 3, 2, 10, \mathbf{e}_{24060})$,
 $y_1 = 420000$.

Model. Linear predictor $h_\theta(\mathbf{x}) = \theta^\top \mathbf{x}$. Each entry of θ is the marginal price contribution of one feature (e.g. $\$/\text{sqft}$).

Squared loss for one example.

$$\ell(h_\theta(\mathbf{x}_i), y_i) = \frac{1}{2}(\theta^\top \mathbf{x}_i - y_i)^2.$$

Penalizes $\2 over- or under-prediction.

Empirical risk: average squared error over all n houses,

$$F(\theta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2}(\theta^\top \mathbf{x}_i - y_i)^2 + \frac{\lambda}{2} \|\theta\|^2.$$

Training = find the θ minimizing F (ridge regression).

What's common to both. Each example contributes one term $\ell_i = \ell(h_\theta(\mathbf{x}_i), y_i)$. Training averages those terms (and adds a regularizer) to get $F(\theta)$. The optimization problem is exactly the kind of finite-sum convex problem that the rest of this section — SGD and mini-batches — is designed for.

For SVM: same template, hinge loss in place of squared/logistic. The next two slides walk through SVM in detail.

Application 2: spam vs. ham (binary classification).

One training example i = one email.

$y_i = +1$ if email is spam, -1 if ham (legitimate).

Model. Score $h_\theta(\mathbf{x}) = \theta^\top \mathbf{x}$, then convert to a probability:

$$\Pr(\text{spam} \mid \mathbf{x}) = \sigma(\theta^\top \mathbf{x}) = \frac{1}{1 + e^{-\theta^\top \mathbf{x}}}.$$

A large positive entry of θ on the “viagra” feature means that word strongly votes for spam.

Logistic loss for one example.

$$\ell(h_\theta(\mathbf{x}_i), y_i) = \log(1 + e^{-y_i \theta^\top \mathbf{x}_i}).$$

Reading: confident and correct ($y_i \theta^\top \mathbf{x}_i \gg 0$)
 $\Rightarrow$ loss $\rightarrow 0$. Wrong or uncertain
($y_i \theta^\top \mathbf{x}_i \leq 0$) $\Rightarrow$ loss is large.

Training = minimize

$$F(\theta) = \frac{1}{n} \sum_i \ell_i + \frac{\lambda}{2} \|\theta\|^2 \text{ (regularized logistic regression)}.$$

Hard-margin SVM, derived from margin maximization

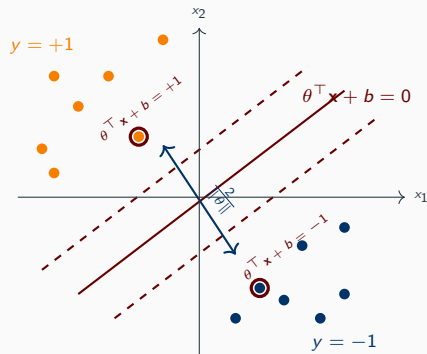
Setup. Binary labels $y_i \in \{-1, +1\}$. Linear classifier $h(\mathbf{x}) = \theta^\top \mathbf{x} + b$. Decision hyperplane $H = \{\mathbf{x} : \theta^\top \mathbf{x} + b = 0\}$.

Step 1: signed distance from $\mathbf{x}_i$ to H .

$$d_i = \frac{\theta^\top \mathbf{x}_i + b}{\|\theta\|}.$$

Multiply by the label y_i to get the **geometric margin** of point i (positive iff $\mathbf{x}_i$ is correctly classified):

$$\gamma_i = \frac{y_i (\theta^\top \mathbf{x}_i + b)}{\|\theta\|}.$$



Maroon: decision boundary. Dashed: the two margin hyperplanes (functional margin ± 1). Width $2/\|\theta\|$. Circled points: **support vectors** — the only ones with the constraint active at the optimum.

Step 2: maximize the smallest margin.

$$\max_{\theta, b} \min_i \frac{y_i (\theta^\top \mathbf{x}_i + b)}{\|\theta\|}.$$

Step 3: rescaling kills one degree of freedom. For any $\alpha > 0$, (θ, b) and $(\alpha\theta, \alpha b)$ define the same hyperplane. Fix the scale by demanding $\min_i y_i (\theta^\top \mathbf{x}_i + b) = 1$. Then the geometric margin equals $1/\|\theta\|$.

Step 4: rewrite as a minimization. Maximizing $1/\|\theta\|$ s.t. functional margin ≥ 1 is the same as

$$\min_{\theta, b} \frac{1}{2} \|\theta\|^2 \quad \text{s.t.} \quad y_i (\theta^\top \mathbf{x}_i + b) \geq 1 \quad \forall i.$$

A convex quadratic program in (θ, b) . The $\frac{1}{2}$ is just for a clean gradient θ .

Soft-margin SVM: two equivalent formulations

When the data is *not* linearly separable the constraint $y_i (\theta^\top \mathbf{x}_i + b) \geq 1$ is infeasible. There are two standard ways to relax it.

Form 1: Slack variables [Cortes & Vapnik 1995, *Machine Learning* 20, "Support-vector networks"]. Introduce non-negative slacks $\xi_i \geq 0$ and penalize them in the objective:

$$\min_{\theta, b, \xi} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i (\theta^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i.$$

ξ_i measures *how badly* point i violates the margin constraint. The penalty parameter $C > 0$ controls the trade-off: large C forces few violations; small C tolerates many.

Form 2: Hinge loss. For fixed (θ, b) , the optimal slack is the smallest non-negative number satisfying the constraint:

$$\xi_i^* = \max\{0, 1 - y_i (\theta^\top \mathbf{x}_i + b)\}.$$

Substitute back into the objective: the slack variables disappear, and we get an unconstrained problem:

$$\min_{\theta, b} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \max\{0, 1 - y_i (\theta^\top \mathbf{x}_i + b)\}.$$

Reparameterizing $\lambda = 1/(nC)$ and dividing by nC gives the form on the previous loss-functions slide:

$$\min_{\theta, b} \frac{1}{n} \sum_{i=1}^n \max\{0, 1 - y_i (\theta^\top \mathbf{x}_i + b)\} + \frac{\lambda}{2} \|\theta\|^2.$$

Two views, one problem.

- Form 1 is a *constrained convex QP* with $n + p + 1$ variables — what off-the-shelf QP solvers (and the SVM dual) consume.
- Form 2 is an *unconstrained, convex, non-smooth* problem — what you write when you want SGD, subgradient methods, or proximal methods.

Solved in practice with: subgradient methods, proximal methods, the SMO algorithm on the dual, or (most often today) stochastic gradient descent on Form 2 — next section.

The SVM gradient as a sum of per-example terms

Objective (hinge-loss form).

$$F(\theta, b) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta, b) + \frac{\lambda}{2} \|\theta\|^2, \quad \ell_i(\theta, b) = \max\{0, 1 - y_i(\theta^\top \mathbf{x}_i + b)\}.$$

Per-example (sub-)gradient. The hinge has a kink at $y_i(\theta^\top \mathbf{x}_i + b) = 1$:

$$\nabla_{\theta} \ell_i = \begin{cases} -y_i \mathbf{x}_i & \text{if } y_i(\theta^\top \mathbf{x}_i + b) < 1 \quad (\text{margin violated}), \\ \mathbf{0} & \text{if } y_i(\theta^\top \mathbf{x}_i + b) > 1 \quad (\text{comfortably correct}), \end{cases} \quad \nabla_b \ell_i = \begin{cases} -y_i & \text{violated} \\ 0 & \text{else.} \end{cases}$$

“Margin violated” means inside the margin strip or misclassified; “comfortably correct” means outside the margin. At the kink, any convex combination of the two sides is a valid subgradient.

Full gradient: a sum over the violating set. Let $\mathcal{V}(\theta, b) = \{i : y_i(\theta^\top \mathbf{x}_i + b) < 1\}$. Then

$$\begin{aligned} \nabla_{\theta} F(\theta, b) &= -\frac{1}{n} \sum_{i \in \mathcal{V}} y_i \mathbf{x}_i + \lambda \theta \\ \nabla_b F(\theta, b) &= -\frac{1}{n} \sum_{i \in \mathcal{V}} y_i \end{aligned}$$

First sum is the data term over violating examples only; second term $\lambda \theta$ is from the regularizer.

Reading the formula.

- Each violating example i contributes the vector $-y_i \mathbf{x}_i / n$ — a tug on θ in the direction that classifies $\mathbf{x}_i$ correctly with more margin.
- Each comfortably-correct example contributes $\mathbf{0}$ — it has no influence on the next step. *Most of the dataset is silent at any given iteration.*
- The regularizer $\lambda \theta$ pulls θ toward $\mathbf{0}$ (smaller norm $\Leftrightarrow$ wider margin).
- At the optimum, $\mathcal{V}$ is exactly the **support-vector set**; the KKT conditions of Form 1 give $\sum_{i \in \mathcal{V}} y_i = 0$, which is just $\nabla_b F = 0$.

Connection to SGD. A single SGD step picks one index i uniformly and updates

$$\theta^{k+1} = (1 - \eta_k \lambda) \theta^k + \eta_k \mathbf{x}_i y_i \mathbb{1}_{[y_i(\theta^k \top \mathbf{x}_i + b^k) < 1]}.$$

This is the SGD update for the regularized hinge loss. Drop the regularizer and shift the margin threshold from 1 to 0, and you get the original **perceptron algorithm** (Rosenblatt 1958) — next slide.

The perceptron algorithm: SGD on a 0-margin hinge loss

Perceptron (Rosenblatt 1958, *Psych. Review* 65; the first machine-learning algorithm). The cleanest example of an SGD-style classifier — predates SGD by a decade and SVM by 35 years.

Generic SGD.

```
Input: data  $\{(x_i, y_i)\}_{i=1}^n$ , objective  $F(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\theta)$ , iterations  $T$ .  
Initialize  $\theta^1 \in \mathbb{R}^d$ .  
For  $k = 1, 2, \dots, T$ :  
1. Sample  $i_k \in \{1, \dots, n\}$ .  
2. Choose step size  $\eta_k$ .  
3. Compute stochastic gradient  $\mathbf{g}_k = \nabla \ell_{i_k}(\theta^k)$ .  
4. Update  $\theta^{k+1} = \theta^k - \eta_k \mathbf{g}_k$ .  
Output:  $\theta^{T+1}$ .
```

Perceptron = SGD with

$$f_i(\theta) = \max\{0, -y_i \theta^\top \mathbf{x}_i\}, \eta_k = 1.$$

```
Input: data  $\{(x_i, y_i)\}_{i=1}^n, y_i \in \{-1, +1\}$ .  
Initialize  $\theta^1 = \mathbf{0}$ .  
For  $k = 1, 2, \dots$ :  
1. Pick  $(x_{i_k}, y_{i_k})$  from the data.  
2. (no choice — step size is fixed at 1.)  
3. Compute the gradient:  

$$\mathbf{g}_k = \begin{cases} -y_{i_k} \mathbf{x}_{i_k} & \text{if } y_{i_k} \theta^{k\top} \mathbf{x}_{i_k} \leq 0 \\ \mathbf{0} & \text{otherwise.} \end{cases}$$
  
4. Update  $\theta^{k+1} = \theta^k - \mathbf{g}_k$ .  
If misclassified:  $\theta^{k+1} = \theta^k + y_{i_k} \mathbf{x}_{i_k}$ .  
else:  $\theta^{k+1} = \theta^k$ .  
Stop when no example is misclassified.
```

The perceptron loss. The function $f_i(\theta) = \max\{0, -y_i \theta^\top \mathbf{x}_i\}$ is the hinge loss with the margin threshold lowered from 1 to 0. Penalty is zero whenever the example is on the right side of the hyperplane (no margin requirement). The (sub-)gradient is $-y_i \mathbf{x}_i$ exactly when the example is misclassified, 0 otherwise — which is what the algorithm computes in step 3.

Update rule, in words. “If you got example i wrong, push θ in the direction $y_{i_k} \mathbf{x}_{i_k}$ that would have classified it correctly.” If you got it right, do nothing.

Theorem (Novikoff 1962, Symposium on the Mathematical Theory of Automata)

If the data is linearly separable with margin $\gamma > 0$ (i.e. $\exists \theta^*$ with $\|\theta^*\| = 1$ and $y_i \theta^{*\top} \mathbf{x}_i \geq \gamma$ for all i), and $\|\mathbf{x}_i\| \leq R$, then the perceptron makes at most $(R/\gamma)^2$ updates before terminating, regardless of the order in which it sees examples.

Compared with SVM.

- Perceptron: 0-margin hinge, no regularizer, fixed step. Stops as soon as it finds any separator.
- SVM: 1-margin hinge, $\frac{\lambda}{2} \|\theta\|^2$ regularizer. Picks the maximum-margin separator.
- Both are SGD on the same template: hinge-style loss, per-example update step.

Historical arc. Rosenblatt 1958 (perceptron) → Robbins–Monro 1951 abstract framework → Vapnik 1979 (SVM) → modern SGD-for-ML. The same per-example update has been re-discovered in three communities under three names.

Lagrangian duality for SVM (1 of 2): the Lagrangian and KKT

Soft-margin primal (Form 1, recap):

$$\min_{\theta, b, \xi} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\theta^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i.$$

Convex QP in (θ, b, ξ) . We have $2n$ inequality constraints, so we'll need $2n$ Lagrange multipliers.

Form the Lagrangian. Multipliers $\alpha_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$:

$$\mathcal{L}(\theta, b, \xi; \alpha, \mu) = \frac{1}{2} \|\theta\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\theta^\top \mathbf{x}_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

KKT stationarity. At the primal optimum $\partial \mathcal{L} / \partial (\text{primal}) = 0$:

$$\begin{aligned} \partial \mathcal{L} / \partial \theta = 0 &\implies \theta^* = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i && (\theta^* \text{ lies in } \text{span}\{\mathbf{x}_i\}) \\ \partial \mathcal{L} / \partial b = 0 &\implies \sum_{i=1}^n \alpha_i y_i = 0 && (\text{equality constraint on } \alpha) \\ \partial \mathcal{L} / \partial \xi_i = 0 &\implies \alpha_i + \mu_i = C, \quad \mu_i \geq 0 \implies 0 \leq \alpha_i \leq C. \end{aligned}$$

KKT complementary slackness. For each i :

$$\alpha_i [y_i(\theta^{*\top} \mathbf{x}_i + b^*) - 1 + \xi_i^*] = 0, \quad (C - \alpha_i) \xi_i^* = 0.$$

Three cases per example.

- $\alpha_i^* = 0$: comfortably correct ($\xi_i^* = 0$, margin > 1). *Not* a support vector.
- $0 < \alpha_i^* < C$: *on the margin* ($\xi_i^* = 0$ and $y_i(\theta^{*\top} \mathbf{x}_i + b^*) = 1$). “Margin support vector”.
- $\alpha_i^* = C$: violator (margin constraint active with $\xi_i^* \geq 0$). “Bounded support vector”.

Lagrangian duality for SVM (2 of 2): substitute back to get the dual

Plug $\theta^* = \sum_i \alpha_i y_i \mathbf{x}_i$, $\sum_i \alpha_i y_i = 0$, and $\mu_i = C - \alpha_i$ into $\mathcal{L}$. Three simplifications:

(i) Quadratic term in θ . Since $\theta^* = \sum_i \alpha_i y_i \mathbf{x}_i$,

$$\frac{1}{2} \|\theta^*\|^2 - \sum_i \alpha_i y_i \theta^{*\top} \mathbf{x}_i = \frac{1}{2} \|\theta^*\|^2 - \theta^{*\top} \theta^* = -\frac{1}{2} \|\theta^*\|^2 = -\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle.$$

(ii) Bias terms. $\sum_i \alpha_i y_i b = b \sum_i \alpha_i y_i = 0$ (using $\sum_i \alpha_i y_i = 0$).

(iii) Slack terms. $C \sum_i \xi_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i = \sum_i (C - \alpha_i - \mu_i) \xi_i = 0$ (stationarity in ξ_i).

Dual function. The remaining terms in $\mathcal{L}$ collapse to

$$g(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle.$$

The SVM dual problem

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C \quad \forall i, \quad \sum_{i=1}^n \alpha_i y_i = 0.$$

Why the dual is useful. A convex quadratic program in $\alpha \in \mathbb{R}^n$, with one equality constraint and box bounds. The data $\mathbf{x}_i$ enter *only* through inner products $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ — the $n \times n$ Gram matrix. Slater's condition holds, so strong duality: solving the dual gives the same optimum as the primal.

Recovering the primal. Once α^* is found:

$$\theta^* = \sum_i \alpha_i^* y_i \mathbf{x}_i, \quad b^* = y_j - \sum_i \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad (\text{any margin SV } j \text{ with } 0 < \alpha_j^* < C).$$

Predictor on a new $\mathbf{x}$.

$$h(\mathbf{x}) = \theta^{*\top} \mathbf{x} + b^* = \sum_{i: \alpha_i^* > 0} \alpha_i^* y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + b^*.$$

Inner products everywhere — this is the wedge that lets us insert kernels.

The kernel trick: from inner products to kernels

The motivation. Many real datasets aren't linearly separable in the original feature space $\mathbb{R}^p$, but become separable after a non-linear map

$$\phi : \mathbb{R}^p \rightarrow \mathbb{R}^q, \quad \mathbf{x} \mapsto \phi(\mathbf{x}),$$

where q may be very large or even infinite. We saw on the previous two slides that the dual SVM and the predictor only ever see the data through inner products. This is the wedge.

The trick. Define a *kernel function*

$$K(\mathbf{x}, \mathbf{x}') := \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle.$$

Replace every inner product in the dual and in the predictor with $K(\cdot, \cdot)$:

$$\boxed{\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \sum_i \alpha_i y_i = 0.}$$

$$h(\mathbf{x}) = \sum_{i: \alpha_i^* > 0} \alpha_i^* y_i K(\mathbf{x}_i, \mathbf{x}) + b^*.$$

What this is doing. We're solving an SVM *exactly* in $\mathbb{R}^q$. The dual problem is the SVM dual on the lifted data $\{\phi(\mathbf{x}_i)\}$. The primal θ now lives in $\mathbb{R}^q$ and is $\sum_i \alpha_i^* y_i \phi(\mathbf{x}_i)$ — but **we never write ϕ down**. As long as K can be evaluated cheaply ($O(p)$ for polynomial / RBF kernels), the algorithm runs in the original $\mathbb{R}^p$ but learns the separator in $\mathbb{R}^q$.

The decision boundary in original space. $h(\mathbf{x}) = 0$ is a *non-linear* curve in $\mathbb{R}^p$: a polynomial of degree d for the polynomial kernel, a smooth contour for the RBF kernel. This is how we got the cluster-vs-background separators on the SVM video slide.

Kernel examples and Mercer's condition

Standard kernels.

Kernel	Formula	Implicit feature map ϕ
Linear	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$	$\phi(\mathbf{x}) = \mathbf{x}$ (no lift)
Polynomial	$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d$	all monomials of total degree $\leq d$
Gaussian (RBF)	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	infinite-dimensional (a Hilbert space of smooth functions)
Sigmoid	$K(\mathbf{x}, \mathbf{x}') = \tanh(\mathbf{a}^\top \mathbf{x}' + c)$	no exact ϕ (not strictly a Mercer kernel)

Worked example: degree-2 polynomial kernel in $\mathbb{R}^2$. Take $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + 1)^2$.

Expanding and matching:

$$K(\mathbf{x}, \mathbf{x}') = 1 + 2x_1x_1' + 2x_2x_2' + x_1^2(x_1')^2 + 2x_1x_2x_1'x_2' + x_2^2(x_2')^2 = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

with the explicit map $\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2) \in \mathbb{R}^6$. SVM in this ϕ -space learns conic decision boundaries (ellipses, hyperbolas) in the original $\mathbf{x}$ -space — without ever building the 6-dimensional features.

When is a function K a valid kernel?

Theorem (Mercer 1909)

A continuous symmetric function K corresponds to an inner product $\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$ for some feature map ϕ if and only if for every finite set $\{\mathbf{x}_1, \dots, \mathbf{x}_m\}$ the Gram matrix $[K(\mathbf{x}_i, \mathbf{x}_j)]_{i,j=1,\dots,m}$ is positive semidefinite.

Calculus of valid kernels. If K_1, K_2 are valid:

- $aK_1 + bK_2$ is valid for $a, b \geq 0$.
- $K_1 \cdot K_2$ is valid (pointwise product).
- $f(\mathbf{x})K_1(\mathbf{x}, \mathbf{x}')f(\mathbf{x}')$ is valid for any function f .
- $K_1(g(\mathbf{x}), g(\mathbf{x}'))$ is valid for any map g .

This gives a constructive way to build new kernels from old ones.

Practical takeaways.

- RBF is the most common production choice. The bandwidth γ is the main hyperparameter; cross-validate.
- Kernel SVM scales as $O(n^2)$ in memory (the Gram matrix) and $O(n^2) - O(n^3)$ in training time. Beyond $n \sim 10^5$, kernel SVMs become impractical — use random Fourier features (Rahimi-Recht 2007) or stick with linear SVM via Pegasos.
- Reading: Schölkopf & Smola, *Learning with Kernels* (MIT Press 2002), Chapters 1–4 for the basics, Chapter 6 for kernel-SVM training algorithms (SMO).

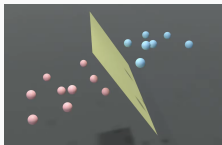
Companion viewing for SVM

Two short videos for further intuition on SVM:

1. Support vector machines, visualized.

youtube.com/watch?v=_YPSrcrkx28

A geometric walkthrough of the maximum-margin idea, the support vectors, and how the soft-margin slack variables relax the hard constraint. Same diagram you saw on the previous two slides, animated.



The margin hyperplane separating two classes.

2. The kernel trick.

youtube.com/watch?v=Q7vT0--5VII

Beyond linear separators. If the classes aren't linearly separable in the original space $\mathbb{R}^p$, lift them implicitly into a higher-dimensional space via a kernel $K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$. The optimization problem only uses inner products, so ϕ is never computed.



*One class (pink) cluster surrounded by the other (blue)
— not linearly separable in 2D, becomes separable
after the kernel lift.*

Reading. Schölkopf & Smola, *Learning with Kernels* (MIT Press 2002), Chapters 1–2 for the basics. The kernel trick connects to the dual form of the soft-margin SVM (Form 1 from the soft-margin slide); the SMO algorithm is the standard production solver in libsvm and scikit-learn.

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

Why SGD: the finite-sum problem

Setup. Most ML problems have the structure

$$\min_{\theta} F(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta),$$

where $f_i(\theta) = \ell(h_{\theta}(\mathbf{x}_i), y_i)$. Full gradient cost: $O(n)$ per step.

SGD (Robbins–Monro 1951). Pick index i_k uniformly. Use the cheap stochastic gradient $\mathbf{g}_k := \nabla f_{i_k}(\theta^k)$. Update:

$$\theta^{k+1} = \theta^k - \eta_k \mathbf{g}_k.$$

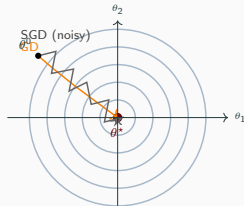
Cost per iteration: $O(1)$ instead of $O(n)$.

Unbiased-gradient property.

$$\mathbb{E}[\mathbf{g}_k \mid \theta^k] = \nabla F(\theta^k).$$

A noisy but *unbiased* estimator. **This is what makes the proofs go through.**

Variance. Define $\sigma^2 := \sup_{\theta} \mathbb{E}[\|\mathbf{g}_k - \nabla F(\theta)\|^2]$ (the gradient noise). The convergence rate of SGD scales with σ^2 .



Same start. GD takes one expensive step toward the optimum; SGD takes many cheap noisy steps with the same total cost.

SGD via the descent lemma: smooth F

Setting. F is L -smooth (each f_i is, so F is). Run SGD with constant step $\eta = 1/L$ and unbiased gradient $\mathbb{E}[\mathbf{g}_k \mid \theta^k] = \nabla F(\theta^k)$, variance $\mathbb{E}[\|\mathbf{g}_k - \nabla F(\theta^k)\|^2 \mid \theta^k] \leq \sigma^2$.

Proof. Apply the descent lemma at θ^k with $\mathbf{y} = \theta^{k+1} = \theta^k - \eta \mathbf{g}_k$:

$$F(\theta^{k+1}) \leq F(\theta^k) + \nabla F(\theta^k)^\top (\theta^{k+1} - \theta^k) + \frac{L}{2} \|\theta^{k+1} - \theta^k\|^2 = F(\theta^k) - \eta \nabla F(\theta^k)^\top \mathbf{g}_k + \frac{L\eta^2}{2} \|\mathbf{g}_k\|^2.$$

(Up to here, the proof is identical to the deterministic GD proof from earlier in the deck. The only thing that's different is that $\mathbf{g}_k$ is a stochastic gradient.)

Take conditional expectation given θ^k . Two stochastic terms become deterministic:

$$\begin{aligned}\mathbb{E}[\nabla F(\theta^k)^\top \mathbf{g}_k \mid \theta^k] &= \nabla F(\theta^k)^\top \mathbb{E}[\mathbf{g}_k \mid \theta^k] = \|\nabla F(\theta^k)\|^2 \quad (\text{unbiasedness}), \\ \mathbb{E}[\|\mathbf{g}_k\|^2 \mid \theta^k] &= \|\nabla F(\theta^k)\|^2 + \mathbb{E}[\|\mathbf{g}_k - \nabla F(\theta^k)\|^2 \mid \theta^k] \leq \|\nabla F(\theta^k)\|^2 + \sigma^2.\end{aligned}$$

The second is the bias-variance decomposition: mean-square = mean² + variance.

Substituting:

$$\mathbb{E}[F(\theta^{k+1}) \mid \theta^k] \leq F(\theta^k) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla F(\theta^k)\|^2 + \frac{L\eta^2}{2} \sigma^2.$$

With $\eta = 1/L$, $\eta(1 - L\eta/2) = 1/(2L)$, so

$$\boxed{\mathbb{E}[F(\theta^{k+1}) \mid \theta^k] \leq F(\theta^k) - \frac{1}{2L} \|\nabla F(\theta^k)\|^2 + \frac{\sigma^2}{2L}.$$

Compare with deterministic GD's per-step decrease $f(\mathbf{x}^{k+1}) \leq f(\mathbf{x}^k) - \frac{1}{2L} \|\nabla f(\mathbf{x}^k)\|^2$. The only change is the additive $\sigma^2/(2L)$ noise term.

Telescope from $k = 0$ to $K - 1$. Take total expectation, rearrange, use $\mathbb{E}[F(\theta^K)] \geq F^*$:

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbb{E}[\|\nabla F(\theta^k)\|^2] \leq \frac{2L(F(\theta^0) - F^*)}{K} + \sigma^2.$$

Theorem (Smooth SGD)

For any L -smooth F and step size $\eta = 1/L$, after K iterations

$$\min_{0 \leq k < K} \mathbb{E}[\|\nabla F(\theta^k)\|^2] \leq \frac{2L(F(\theta^0) - F^*)}{K} + \sigma^2.$$

Reading the bound. Two terms: a deterministic $O(1/K)$ rate (same as GD), plus a constant noise floor σ^2 . To drive the floor down, decrease η (the noise term scales as $L\eta\sigma^2$). To drive the deterministic part down, increase K . The expectation flowed through the descent-lemma proof one step at a time — nothing else changed.

Take-away. The next two slides re-derive convergence for SGD via a different (distance-tracking) route. They drop the smoothness assumption and give clean rates in $\mathbb{E}[F(\bar{\theta}) - F^*]$ instead of gradient norms.

SGD: convergence in expectation, convex case

Theorem (Nemirovski–Juditsky–Lan–Shapiro 2009; Bubeck 2015 §6.1)

Suppose each f_i is convex, F is convex, and the variance bound $\mathbb{E}[\|\mathbf{g}_k\|^2] \leq G^2$ holds for all k .

Run SGD with constant step $\eta = D/(G\sqrt{K})$, where $D = \|\theta^0 - \theta^*\|$, and define the running average $\bar{\theta}^K := \frac{1}{K} \sum_{k=0}^{K-1} \theta^k$. Then

$$\mathbb{E}[F(\bar{\theta}^K) - F^*] \leq \frac{D G}{\sqrt{K}}.$$

Proof. Expand the squared distance using $\theta^{k+1} = \theta^k - \eta \mathbf{g}_k$:

$$\|\theta^{k+1} - \theta^*\|^2 = \|\theta^k - \theta^*\|^2 - 2\eta \mathbf{g}_k^\top (\theta^k - \theta^*) + \eta^2 \|\mathbf{g}_k\|^2.$$

Take conditional expectation given θ^k . The cross term is linear in $\mathbf{g}_k$, so by unbiasedness

$\mathbb{E}[\mathbf{g}_k \mid \theta^k] = \nabla F(\theta^k)$:

$$\mathbb{E}[\mathbf{g}_k^\top (\theta^k - \theta^*) \mid \theta^k] = \nabla F(\theta^k)^\top (\theta^k - \theta^*) \geq F(\theta^k) - F^*, \quad (\text{convexity of } F).$$

The variance term is bounded by hypothesis: $\mathbb{E}[\|\mathbf{g}_k\|^2 \mid \theta^k] \leq G^2$. Substituting,

$$\mathbb{E}[\|\theta^{k+1} - \theta^*\|^2 \mid \theta^k] \leq \|\theta^k - \theta^*\|^2 - 2\eta (F(\theta^k) - F^*) + \eta^2 G^2.$$

Take total expectation, sum from $k = 0$ to $K - 1$, telescope:

$$\sum_{k=0}^{K-1} \mathbb{E}[F(\theta^k) - F^*] \leq \frac{D^2}{2\eta} + \frac{K\eta G^2}{2}.$$

Divide by K , apply Jensen on the convex F to pull the average inside, and optimize

$\eta = D/(G\sqrt{K})$:

$$\mathbb{E}[F(\bar{\theta}^K) - F^*] \leq \frac{1}{K} \sum_k \mathbb{E}[F(\theta^k) - F^*] \leq \frac{D G}{\sqrt{K}}. \quad \square$$

Reading the rate. $O(1/\sqrt{K})$ is the price of stochasticity: SGD on convex functions is *slower* than GD's $O(1/K)$. The trade-off is the per-iteration cost: $O(1)$ vs. $O(n)$.

SGD: convergence in expectation, strongly convex case

Theorem (Polyak & Juditsky 1992; Lacoste-Julien-Schmidt-Bach 2012)

Suppose F is μ -strongly convex and $\mathbb{E}[\|\mathbf{g}_k\|^2] \leq G^2$. Run SGD with diminishing step

$\eta_k = \frac{2}{\mu(k+1)}$. Then

$$\mathbb{E}[\|\theta^k - \theta^*\|^2] \leq \frac{4G^2}{\mu^2 k}.$$

Proof. Strong convexity at θ^k and θ^* gives the key descent inequality

$$\nabla F(\theta^k)^\top (\theta^k - \theta^*) \geq \mu \|\theta^k - \theta^*\|^2.$$

Expand the squared distance using $\theta^{k+1} = \theta^k - \eta_k \mathbf{g}_k$, then take conditional expectation given θ^k . As in the convex proof, unbiasedness $\mathbb{E}[\mathbf{g}_k | \theta^k] = \nabla F(\theta^k)$ converts the cross term:

$$\mathbb{E}[\|\theta^{k+1} - \theta^*\|^2 | \theta^k] = \|\theta^k - \theta^*\|^2 - 2\eta_k \nabla F(\theta^k)^\top (\theta^k - \theta^*) + \eta_k^2 \mathbb{E}[\|\mathbf{g}_k\|^2 | \theta^k].$$

Apply the strong-convexity inequality on the cross term, the variance bound on the last term, and take total expectation. Letting $a_k := \mathbb{E}[\|\theta^k - \theta^*\|^2]$,

$$a_{k+1} \leq (1 - 2\mu\eta_k) a_k + \eta_k^2 G^2.$$

With $\eta_k = 2/(\mu(k+1))$: $1 - 2\mu\eta_k = (k-3)/(k+1)$, so

$$a_{k+1} \leq \frac{k-3}{k+1} a_k + \frac{4G^2}{\mu^2(k+1)^2}.$$

Induction. Claim $a_k \leq 4G^2/(\mu^2 k)$ for $k \geq 4$. Plug in:

$$a_{k+1} \leq \frac{k-3}{k+1} \cdot \frac{4G^2}{\mu^2 k} + \frac{4G^2}{\mu^2(k+1)^2} \leq \frac{4G^2}{\mu^2(k+1)} \left[\frac{k-3}{k} + \frac{1}{k+1} \right] \leq \frac{4G^2}{\mu^2(k+1)},$$

using $\frac{k-3}{k} + \frac{1}{k+1} \leq 1$ for $k \geq 3$. $\square$

Compare with deterministic GD.

- Deterministic GD on μ -strongly convex: $(1 - \mu/L)^k$, linear convergence.
- SGD on μ -strongly convex: $O(1/k)$, sublinear.

The variance σ^2 acts as a noise floor. Linear rates are unavailable to plain SGD.

(Variance-reduced methods like SVRG and SAGA recover linear rates by clever bias correction

— a story for next lecture.)

Mini-batch SGD: between SGD and full-batch GD

Mini-batch. At iteration k , sample a batch $\mathcal{I}_k \subseteq \{1, \dots, n\}$ of size b . Use

$$\mathbf{g}_k = \frac{1}{b} \sum_{i \in \mathcal{I}_k} \nabla f_i(\theta^k).$$

- $b = 1$ recovers vanilla SGD.
- $b = n$ recovers full-batch gradient descent.
- $b = 32, 64, 128, \dots$ in practice (powers of 2 to fit GPU memory layouts).

Statistics of the mini-batch gradient.

$$\begin{aligned} \mathbb{E}[\mathbf{g}_k \mid \theta^k] &= \nabla F(\theta^k) && \text{(unbiased, same as SGD).} \\ \text{Var}(\mathbf{g}_k \mid \theta^k) &= \frac{1}{b} \text{Var}(\nabla f_i(\theta^k)) && \text{(variance shrinks linearly in } b\text{).} \end{aligned}$$

What this buys. The convex-case bound (theorem two slides back) becomes

$$\mathbb{E}[F(\bar{\theta}^K) - F^*] \leq \frac{D G / \sqrt{b}}{\sqrt{K}} = \frac{DG}{\sqrt{bK}}.$$

Doubling b halves the noise term — but doubles the per-iteration cost. *In wall-clock time*, mini-batch is a wash compared with vanilla SGD if FLOPs are linear in b .

Why mini-batches dominate in practice. GPU parallelism. A batch of 64 examples is processed in essentially the same wall-clock time as a single example, up to memory bandwidth limits. Up to that hardware sweet spot, mini-batching gives *free* variance reduction.

Modern practice. SGD with momentum (heavy-ball update on $\mathbf{g}_k$), or Adam (per-coordinate adaptive step sizes computed from EMAs of $\mathbf{g}_k$ and $\mathbf{g}_k^2$). Both build on the mini-batch SGD foundation.

Mini-batch SGD: why, when, and what the theory gives you

Three theoretical improvements from batches of size b . The key fact: $\text{Var}(\mathbf{g}_k) = \sigma^2/b$ (i.i.d. averaging). Plug this into each of the three SGD bounds we proved:

Setting	Vanilla SGD ($b=1$)	Mini-batch ($b>1$)
Smooth, descent-lemma proof	$\min_k \mathbb{E}[\ \nabla F\ ^2] \leq \frac{2L(P^* - F^*)}{\epsilon} + \sigma^2$	$\min_k \mathbb{E}[\ \nabla F\ ^2] \leq \frac{2L(P^* - F^*)}{\epsilon} + \frac{\sigma^2}{b}$
Convex (averaging proof)	$\mathbb{E}[F(\bar{\theta}) - F^*] \leq DG/\sqrt{K}$	$\mathbb{E}[F(\bar{\theta}) - F^*] \leq DG/\sqrt{bK}$
μ -strongly convex	$\mathbb{E}[\ \theta^k - \theta^*\ ^2] \leq 4\sigma^2/(\mu^2 k)$	$\mathbb{E}[\ \theta^k - \theta^*\ ^2] \leq 4\sigma^2/(\mu^2 bk)$

The noise term shrinks by exactly $1/b$ in every bound. The deterministic part (from the gradient) is unchanged.

When does this give a real speedup?

- **Noise-dominated regime.** If σ^2 dominates the bound, then larger b directly reduces wall-clock iterations to a target accuracy: $K_b = K_1/b$.
- **Deterministic-dominated regime.** If the $1/K$ deterministic term dominates, larger b has no benefit and just costs more FLOPs per iteration.
- **The cross-over.** Halfway through training, you typically transition from deterministic-dominated (early, gradients are large) to noise-dominated (late, gradients are small). Bigger batches help most near the end.

Linear scaling rule [Goyal et al. 2017, "Accurate, Large Minibatch SGD"]: when increasing b by a factor c , also increase the learning rate η by c . Justification: c small steps of size η approximate one big step of size $c\eta$ on a batch of size cb . Empirically holds up to a problem-dependent batch size B^* (the **critical batch size**).

Critical batch size [McCandlish et al. 2018, "An Empirical Model of Large-Batch Training"]. Speedup saturates beyond

$$B^* \approx \frac{\text{tr}(H) \sigma^2}{\|\nabla F\|^2},$$

where H is the Hessian at the current iterate. Small far from minima, large near them. For ImageNet-scale problems, $B^* \sim 10^3$ – 10^4 . Beyond B^* , the linear-speedup picture breaks — you start needing per-coordinate adaptation (Adam) or higher-order corrections.

Practical guidance.

- **Default:** $b \in \{32, 64, 128\}$. Doubles fit well in GPU memory and parallelism.
- **Scaling up:** when increasing b for distributed / large-scale training, apply the linear scaling rule and add a learning-rate *warmup* (linear η ramp over the first ~ 5 epochs).
- **Watching for failure:** if loss plateaus or diverges, you've likely passed B^* . Reduce b or use Adam.

Reading: Bottou, Curtis & Nocedal (2018), Optimization Methods for Large-Scale Machine Learning, SIAM Review 60(2): comprehensive survey of mini-batch theory and practice in modern ML, with the variance-reduction calculus done carefully. Smith, Kindermans, Ying & Le (2018), Don't Decay the Learning Rate, Increase the Batch Size: the practical observation that the linear scaling rule lets you trade learning-rate decay for batch-size growth.

Companion viewing for SGD

Short video for further intuition:

Stochastic gradient descent.

youtube.com/watch?v=UmathvAKj80

A geometric walkthrough of how SGD works on a finite-sum loss $F(\theta) = \frac{1}{n} \sum_i f_i(\theta)$: each iteration picks one example, takes a noisy step, and the trajectory wanders toward the minimum rather than driving straight at it.

What to watch for.

- The iterates jitter even near the optimum — that's the variance term σ^2 from the convex-case rate proof we just did.
- Mini-batches damp the jitter without changing the direction of progress (variance scales as σ^2/b).
- With a diminishing step size ($\eta_k \rightarrow 0$), the jitter shrinks with k and the iterates settle on the optimum — this is exactly the $O(1/k)$ rate from the strongly-convex theorem.

Same empirical-risk-minimization framing as the ML warm-up section, so the loss landscape in the video is exactly $F(\theta)$ as defined on the slides.

Roadmap

Setup and Notation

Recap: Gradient Descent and Conjugate Gradient

Heavy-Ball Momentum

Nesterov Acceleration

Machine Learning Warm-Up

Stochastic Gradient Descent

Summary

The four algorithms, one slide

Method	Update	When to reach for it
Gradient descent	$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f(\mathbf{x}^k)$	Anything smooth; easy and robust.
Conjugate gradient	$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{p}^k, \mathbf{p}^k = \mathbf{r}_k + \beta_k \mathbf{p}^{k-1}$	Quadratic / linear-system problems.
Heavy-ball	$\mathbf{x}^{k+1} = \mathbf{x}^k - \eta \nabla f(\mathbf{x}^k) + \beta(\mathbf{x}^k - \mathbf{x}^{k-1})$	Practical default with momentum, esp. in DL.
Nesterov	$\mathbf{x}^{k+1} = \mathbf{y}^k - \frac{1}{L} \nabla f(\mathbf{y}^k), \mathbf{y}^k = \mathbf{x}^k + \beta_k(\mathbf{x}^k - \mathbf{x}^{k-1})$	First-order optimum on smooth convex.

Mental model.

- GD: pure local information, no memory. Works always but slow.
- CG: optimal momentum, computed adaptively, exploits the quadratic structure. *Solves quadratics.*
- Heavy-ball: momentum with fixed coefficients. *Speeds up quadratics; doesn't accelerate the convex rate.*
- Nesterov: momentum with scheduled coefficient and a gradient look-ahead. *Optimal on convex smooth, period.*

Where to read each result in Beck and Nocedal–Wright

Slide / theorem	Source
Descent lemma	Beck, Lemma 5.7
GD per-step decrease	Beck, Lemma 10.4 (with $g \equiv 0$)
GD $O(1/k)$ rate (convex)	Beck, Theorem 10.21 (with $g \equiv 0$, $\alpha = 1$)
GD linear rate (strongly convex)	Beck, Theorem 10.29 (with $g \equiv 0$)
Quadratic problem, CG setup	N&W, §5.1
A-conjugate directions, finite termination	N&W, Theorem 5.1 (conjugate-direction theorem)
CG algorithm (Hestenes–Stiefel)	N&W, Algorithm 5.2
CG $\sqrt{\kappa}$ rate via Krylov subspace	N&W, Theorems 5.3, 5.5
Polyak heavy-ball, quadratic rate	Polyak (1964); N&W, p. 78 (briefly)
Nesterov $O(1/k^2)$ rate (convex)	Beck, Theorem 10.34 (proof in §10.7.2)
Nesterov $\sqrt{\kappa}$ rate (strongly convex)	Beck, Theorem 10.42 (V-FISTA)
Lower bound (no FOM beats $O(1/k^2)$)	Nesterov 2004, §2.1.2

One-line proof-vs-Beck audit: every theorem statement on these slides matches Beck verbatim. The strongly-convex GD proof on the slide uses the Polyak–Łojasiewicz inequality; Beck instead derives the same rate from the prox-grad inequality directly. Either route gets the contraction constant $(1 - \mu/L)$. The Nesterov potential function on the slide is Beck’s Φ_k written with the same off-by-one convention as in his §10.7.2.

References

Books.

- **Nocedal & Wright**, *Numerical Optimization*, 2nd ed., Springer 2006.
The reference for GD and CG. Ch. 3 (line search), Ch. 5 (CG, with all proofs).
- **Beck**, *First-Order Methods in Optimization*, SIAM 2017.
The reference for Nesterov / FISTA. Lemma 5.7 (descent), Theorems 10.21, 10.29 (GD rates as special cases), Theorems 10.34, 10.42 (Nesterov rates).
- **Nesterov**, *Introductory Lectures on Convex Optimization*, Springer 2004.
Original treatment with estimating sequences (§2.2) and the lower bound (§2.1.2).
- **Bubeck**, *Convex Optimization: Algorithms and Complexity*, FnT ML 8(3–4), 2015.
Compact, free monograph. Best survey-length treatment of accelerated methods.

Papers worth knowing.

- Hestenes & Stiefel (1952), *Methods of conjugate gradients for solving linear systems*, J. Res. NBS.
- Polyak (1964), *Some methods of speeding up the convergence of iteration methods*, USSR Comp. Math. (heavy-ball).
- Nesterov (1983), *A method of solving a convex programming problem with convergence rate $O(1/k^2)$* , Soviet Math. Dokl.